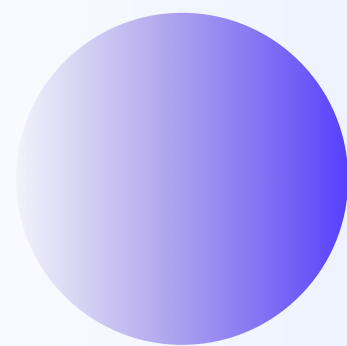
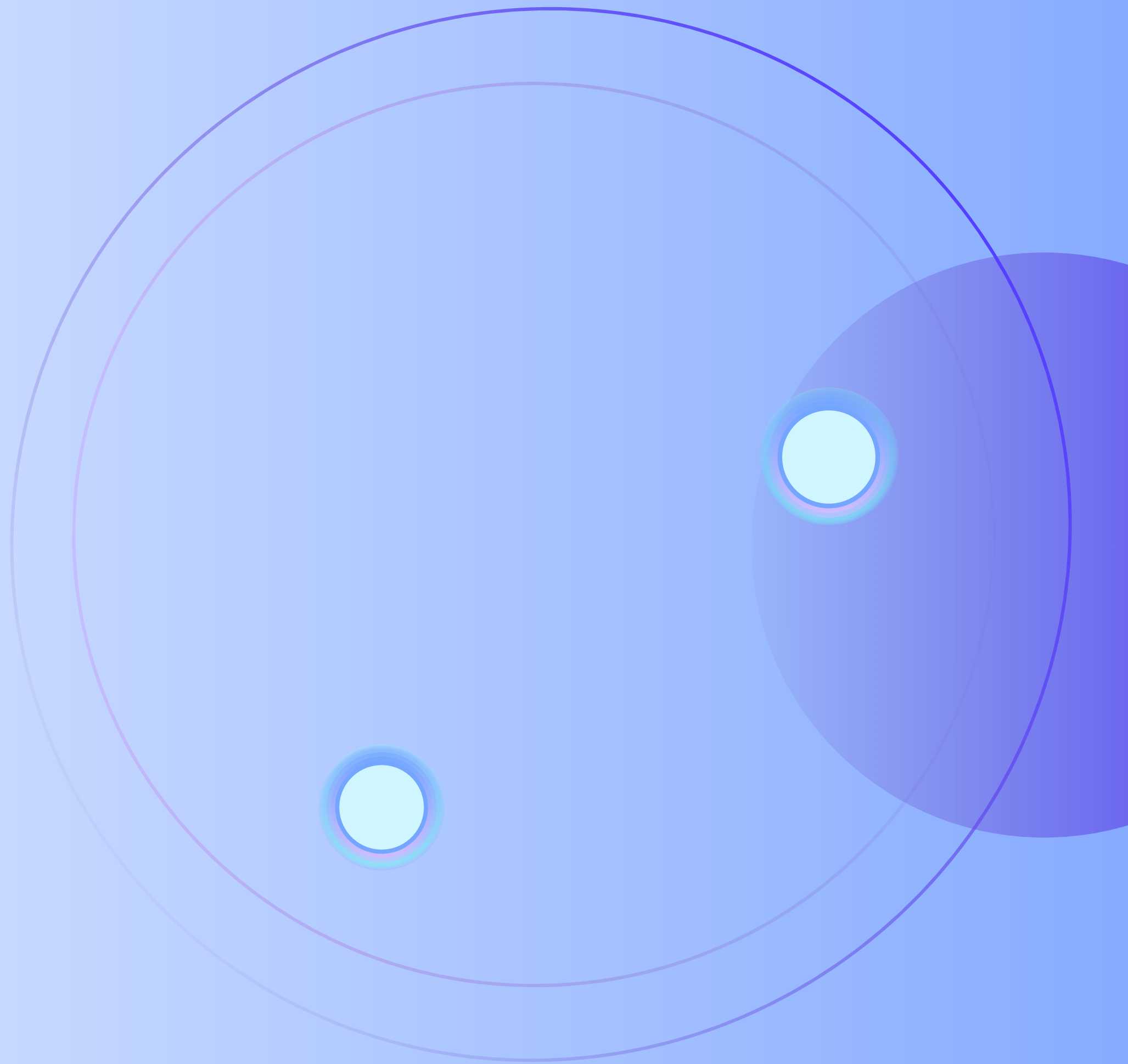




TO-DO LIST

REMAS ALIESSA
KADI ALSUBAIE
KADI ALBAKRI



INTRODUCTION:



Our project, “To-Do List Application,” was developed in C++.

It helps users manage tasks by adding, updating, deleting, and marking them as completed.

We used a singly linked list for flexibility and easy updates.



PROJECT IDEA (PROBLEM)

People often forget tasks without an organized system.

Our project provides a simple TO-DO LIST using a SINGLY LINKED LIST to easily add, edit, delete, and complete tasks.

OBJECTIVES

- **Build a functional To-Do List application.**
- **Apply the linked list data structure to manage tasks dynamically.**
- **Strengthen our skills in C++, classes, pointers, and dynamic memory.**
- **Use data structures to create a practical, user-focused solution.**

CODE

EXPLANATION

- We used a singly linked list to store tasks dynamically and flexibly.
- Each task has:
 - description (string)
 - completed (bool)
 - next (pointer to the next task)

CODE

EXPLANATION

- We designed a class `TodoList` with the following methods:
 - **Constructor:** Initializes head to NULL.
 - **Destructor:** Frees memory at the end of the program.
 - **addTask:** Adds a new task to the list.
 - **deleteTask:** Deletes a task by position.
 - **markCompleted:** Marks a task as completed.
 - **displayTasks:** Displays all tasks with their status.
 - **updateTask:** Updates a task description

RESULT

```
===== TO-DO LIST MANAGER =====

Menu:
1. Add a new task
2. Delete a task
3. Mark task as completed
4. Update a task
5. View all tasks
6. Exit
Enter your choice: 1
Enter task description: Getting prepared for data structure assignment
Task added successfully!

Menu:
1. Add a new task
2. Delete a task
3. Mark task as completed
4. Update a task
5. View all tasks
6. Exit
Enter your choice: 1
Enter task description: Submitting our project
Task added successfully!

Menu:
1. Add a new task
2. Delete a task
3. Mark task as completed
4. Update a task
5. View all tasks
6. Exit
Enter your choice: 5

----- YOUR TASKS -----
Task 1: Getting prepared for data structure assignment [Not Completed]
Task 2: Submitting our project [Not Completed]
-----
```

```
Menu:
1. Add a new task
2. Delete a task
3. Mark task as completed
4. Update a task
5. View all tasks
6. Exit
Enter your choice: 4
Enter task position to update: 2
Enter new description: Submitting our TO-DO List project
Task "Submitting our project" updated to "Submitting our TO-DO List project".

Menu:
1. Add a new task
2. Delete a task
3. Mark task as completed
4. Update a task
5. View all tasks
6. Exit
Enter your choice: 3
Enter task position to mark as completed: 1
Task "Getting prepared for data structure assignment" is now completed!

Menu:
1. Add a new task
2. Delete a task
3. Mark task as completed
4. Update a task
5. View all tasks
6. Exit
Enter your choice: 5

----- YOUR TASKS -----
Task 1: Getting prepared for data structure assignment [Completed]
Task 2: Submitting our TO-DO List project [Not Completed]
-----
```

```
Menu:
1. Add a new task
2. Delete a task
3. Mark task as completed
4. Update a task
5. View all tasks
6. Exit
Enter your choice: 2
Enter task position to delete: 1
Task "Getting prepared for data structure assignment" has been deleted.

Menu:
1. Add a new task
2. Delete a task
3. Mark task as completed
4. Update a task
5. View all tasks
6. Exit
Enter your choice: 5

----- YOUR TASKS -----
Task 1: Submitting our TO-DO List project [Not Completed]
-----

Menu:
1. Add a new task
2. Delete a task
3. Mark task as completed
4. Update a task
5. View all tasks
6. Exit
Enter your choice: 6
Exiting program. Goodbye!

Process returned 0 (0x0)   execution time : 172.022 s
Press any key to continue.
```

CONCLUSION

- We applied what we learned in class to build a real project.
- We gained experience with dynamic memory, pointers, and class design in C++.
- We learned how data structures solve real-world problems.
- This project improved our technical skills and boosted our confidence.